

# Model Evaluation Guide

Complete guide to evaluating LLMs and AI models.

## Table of Contents

- 1. [LLM Evaluation Methods](#)
- 2. [Automated Evaluation](#)
- 3. [Human Evaluation](#)
- 4. [Benchmark Datasets](#)
- 5. [Code Examples](#)
- 6. [Tools & Frameworks](#)

## LLM Evaluation Methods

### 1. Automated Metrics

#### A. BLEU Score (Text Generation)

**What:** Measures n-gram overlap between generated and reference text

**Use for:** Translation, summarization

```
from nltk.translate.bleu_score import sentence_bleu

reference = [['the', 'cat', 'is', 'on', 'the', 'mat']]
candidate = ['the', 'cat', 'is', 'on', 'the', 'mat']

score = sentence_bleu(reference, candidate)
print(f"BLEU: {score}") # 1.0 = perfect match
```

**Range:** 0-1 (higher = better) **Pros:** Fast, automated **Cons:** Doesn't capture meaning

#### B. ROUGE Score (Summarization)

**What:** Recall-oriented metric for summaries

**Types:**

- ROUGE-N: N-gram overlap
- ROUGE-L: Longest common subsequence
- ROUGE-S: Skip-bigram

```
from rouge_score import rouge_scorer

scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'])

reference = "The cat sat on the mat"
candidate = "A cat was sitting on the mat"

scores = scorer.score(reference, candidate)
print(scores)
# {'rouge1': F(precision=0.857, recall=0.857, fmeasure=0.857), ...}
```

**Use for:** Summarization tasks

---

## C. Perplexity (Language Modeling)

**What:** How "surprised" model is by text (lower = better)

```
from transformers import GPT2LMHeadModel, GPT2Tokenizer
import torch

model = GPT2LMHeadModel.from_pretrained('gpt2')
tokenizer = GPT2Tokenizer.from_pretrained('gpt2')

text = "The quick brown fox"
tokens = tokenizer.encode(text, return_tensors='pt')

with torch.no_grad():
    outputs = model(tokens, labels=tokens)
    loss = outputs.loss
    perplexity = torch.exp(loss)

print(f"Perplexity: {perplexity.item()}")
# Lower = better (model is less surprised)
```

**Range:** 0- $\infty$  (lower = better) **Use for:** Comparing language models

---

## D. BERTScore (Semantic Similarity)

**What:** Uses BERT embeddings to measure semantic similarity

```
from bert_score import score

candidates = ["The cat sat on the mat"]
references = ["A cat was sitting on the mat"]
```

```
P, R, F1 = score(candidates, references, lang='en')
print(f"F1: {F1.mean()}") # 0.95 (high semantic similarity)
```

**Range:** 0-1 (higher = better) **Use for:** When meaning matters more than exact words

---

## 2. Task-Specific Evaluation

### A. Classification Accuracy

```
from sklearn.metrics import accuracy_score, classification_report

y_true = [0, 1, 1, 0, 1]
y_pred = [0, 1, 0, 0, 1]

accuracy = accuracy_score(y_true, y_pred)
print(f"Accuracy: {accuracy}") # 0.8

print(classification_report(y_true, y_pred))
```

#### Metrics:

- Accuracy: Correct / Total
  - Precision: True Positives / (TP + False Positives)
  - Recall: True Positives / (TP + False Negatives)
  - F1: Harmonic mean of precision & recall
- 

### B. Exact Match (QA Systems)

```
def exact_match(predicted, ground_truth):
    return 1 if predicted.strip().lower() == ground_truth.strip().lower() else 0

predicted = "Paris"
ground_truth = "Paris"
score = exact_match(predicted, ground_truth) # 1
```

**Use for:** Factual QA, named entity recognition

---

### C. Token-level Accuracy (NER)

```
from seqeval.metrics import classification_report

y_true = [['O', 'B-PER', 'I-PER', 'O', 'B-LOC']]
y_pred = [['O', 'B-PER', 'I-PER', 'O', 'B-LOC']]
```

```
print(classification_report(y_true, y_pred))
```

**Use for:** Named Entity Recognition, POS tagging

### 3. Human Evaluation Criteria

#### A. Likert Scale (1-5)

Rate the response quality:

1 - Very Poor

2 - Poor

3 - Acceptable

4 - Good

5 - Excellent

#### B. Pairwise Comparison

Which response is better?

☐ Response A

☐ Response B

☐ Tie

#### C. Multi-Aspect Evaluation

Relevance:

☆☆☆☆☆

Coherence:

☆☆☆☆☆

Fluency:

☆☆☆☆☆

Factuality:

☆☆☆☆☆

## Automated Evaluation Frameworks

### 1. LangChain Evaluation

```
from langchain.evaluation import load_evaluator
from langchain.chat_models import ChatOpenAI

# QA Correctness
evaluator = load_evaluator("qa")
result = evaluator.evaluate_strings(
    prediction="Paris",
```

```
        reference="Paris",
        input="What is the capital of France?"
    )
    print(result)  # {'score': 1.0}

# Criteria-based (requires LLM)
evaluator = load_evaluator(
    "criteria",
    criteria="helpfulness",
    llm=ChatOpenAI(model="gpt-4")
)
result = evaluator.evaluate_strings(
    prediction="You can find that on Wikipedia",
    input="What is the capital of France?"
)
print(result)
# {'reasoning': '...', 'value': 'N', 'score': 0}
```

---

## 2. OpenAI Evals

```
# Install: pip install evals

# Create eval.yaml
"""
custom_eval:
  class: evals.elsuite.basic.match:Match
  args:
    samples_jsonl: samples.jsonl
"""

# samples.jsonl
"""
{"input": "2+2", "ideal": "4"}
{"input": "Capital of France", "ideal": "Paris"}
"""

# Run: oaieval gpt-4 custom_eval
```

---

## 3. RAGAS (RAG Evaluation)

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness,
    answer_relevancy,
    context_precision,
    context_recall
)
```

```
dataset = {
    'question': ['What is Paris?'],
    'answer': ['Paris is the capital of France'],
    'contexts': [['Paris is a city in France']],
    'ground_truth': ['Capital of France']
}

result = evaluate(
    dataset,
    metrics=[
        faithfulness,
        answer_relevancy,
        context_precision,
        context_recall
    ]
)
print(result)
```

---

## 4. TruLens (Observability)

```
from trulens_eval import TruChain, Feedback
from trulens_eval.feedback import Groundedness

# Define feedback functions
grounded = Groundedness()

f_groundness = Feedback(
    grounded.groundedness_measure_with_cot_reasons
).on_input_output()

# Wrap your chain
tru_chain = TruChain(
    chain,
    app_id='my_app',
    feedbacks=[f_groundness]
)

# Run and auto-evaluate
result = tru_chain("What is AI?")
```

---

## Benchmark Datasets

### 1. General Language Understanding

#### GLUE (General Language Understanding Evaluation)

- Tasks: Sentiment, entailment, similarity

- 9 tasks total
- Leaderboard: <https://gluebenchmark.com/>

```
from datasets import load_dataset

# Load GLUE task
dataset = load_dataset("glue", "sst2") # Sentiment

print(dataset['train'][0])
# {'sentence': 'hide new secretions...', 'label': 0}
```

### SuperGLUE

- Harder version of GLUE
  - 8 tasks
- 

## 2. Question Answering

### SQuAD (Stanford QA Dataset)

```
from datasets import load_dataset

dataset = load_dataset("squad")
print(dataset['train'][0])
# {
#   'context': 'Architecturally, the school has...',
#   'question': 'To whom did the Virgin Mary...',
#   'answers': {'text': ['Saint Bernadette'], ...}
# }
```

### Natural Questions (Google)

- Real Google search queries
  - Long-form answers
- 

## 3. Code Generation

### HumanEval

```
from datasets import load_dataset

dataset = load_dataset("openai_humaneval")
print(dataset['test'][0])
# {
#   'task_id': 'HumanEval/0',
#   'prompt': 'def has_close_elements(numbers, threshold):\n    """\n        Check
```

```
if...',
# 'test': 'def check(candidate):\n    assert candidate([1.0, 2.0], 0.3) ==
False',
# }
```

### MBPP (Mostly Basic Python Problems)

- 974 Python programming problems
- 

## 4. Reasoning

### GSM8K (Math Word Problems)

```
dataset = load_dataset("gsm8k", "main")
print(dataset['train'][0])
# {
#   'question': 'Natalia sold clips to 48 friends...',
#   'answer': '###\nStep 1: 48/2 = 24...'
# }
```

### ARC (AI2 Reasoning Challenge)

- Science questions
  - Easy & Challenge sets
- 

## Practical Evaluation Pipeline

### Complete Example: Evaluate Text Generation

```
import openai
from bert_score import score
from rouge_score import rouge_scorer
import numpy as np

class LMEvaluator:
    def __init__(self):
        self.rouge_scorer = rouge_scorer.RougeScorer(
            ['rouge1', 'rouge2', 'rougeL'],
            use_stemmer=True
        )

    def evaluate_single(self, generated, reference):
        """Evaluate single generation"""

        # 1. ROUGE scores
        rouge = self.rouge_scorer.score(reference, generated)
```



```

# 2. BERTScore
P, R, F1 = score([generated], [reference], lang='en')

# 3. Exact match
exact = 1 if generated.strip() == reference.strip() else 0

return {
    'rouge1': rouge['rouge1'].fmeasure,
    'rouge2': rouge['rouge2'].fmeasure,
    'rougeL': rouge['rougeL'].fmeasure,
    'bert_f1': F1.item(),
    'exact_match': exact
}

def evaluate_batch(self, test_cases):
    """Evaluate multiple test cases"""
    results = []

    for case in test_cases:
        prompt = case['prompt']
        reference = case['reference']

        # Generate
        response = openai.ChatCompletion.create(
            model="gpt-3.5-turbo",
            messages=[{"role": "user", "content": prompt}]
        )
        generated = response.choices[0].message.content

        # Evaluate
        metrics = self.evaluate_single(generated, reference)
        metrics['prompt'] = prompt
        metrics['generated'] = generated
        metrics['reference'] = reference

        results.append(metrics)

    return results

def aggregate_metrics(self, results):
    """Calculate average metrics"""
    metrics = ['rouge1', 'rouge2', 'rougeL', 'bert_f1', 'exact_match']

    aggregated = {}
    for metric in metrics:
        values = [r[metric] for r in results]
        aggregated[f'{metric}_mean'] = np.mean(values)
        aggregated[f'{metric}_std'] = np.std(values)

    return aggregated

# Usage
evaluator = LLMEvaluator()

```

```
test_cases = [
    {
        'prompt': 'What is the capital of France?',
        'reference': 'Paris'
    },
    {
        'prompt': 'Write a hello world in Python',
        'reference': 'print("Hello, World!")'
    }
]

results = evaluator.evaluate_batch(test_cases)
summary = evaluator.aggregate_metrics(results)

print("Results:")
for r in results:
    print(f"Prompt: {r['prompt']}")
    print(f"Generated: {r['generated']}")
    print(f"ROUGE-1: {r['rouge1']:.3f}")
    print(f"BERTScore: {r['bert_f1']:.3f}")
    print()

print("\nAggregate Metrics:")
for metric, value in summary.items():
    print(f"{metric}: {value:.3f}")
```

---

## A/B Testing LLMs

### Compare Two Models

```
class ModelComparison:
    def __init__(self, model_a, model_b):
        self.model_a = model_a
        self.model_b = model_b

    def run_comparison(self, test_cases):
        """Compare two models on same inputs"""
        results = []

        for case in test_cases:
            prompt = case['prompt']
            reference = case['reference']

            # Generate from both
            response_a = self.generate(self.model_a, prompt)
            response_b = self.generate(self.model_b, prompt)

            # Evaluate both
            score_a = self.evaluate(response_a, reference)
            score_b = self.evaluate(response_b, reference)
```

```
        results.append({
            'prompt': prompt,
            'model_a': response_a,
            'model_b': response_b,
            'score_a': score_a,
            'score_b': score_b,
            'winner': 'A' if score_a > score_b else 'B'
        })

    return results

def statistical_significance(self, results):
    """Check if difference is statistically significant"""
    from scipy import stats

    scores_a = [r['score_a'] for r in results]
    scores_b = [r['score_b'] for r in results]

    # Paired t-test
    t_stat, p_value = stats.ttest_rel(scores_a, scores_b)

    return {
        't_statistic': t_stat,
        'p_value': p_value,
        'significant': p_value < 0.05
    }
```

---

## Human Evaluation Setup

### 1. Create Evaluation Interface

```
import gradio as gr

def evaluate_response(prompt, response):
    """Human evaluator interface"""

    return gr.Interface(
        fn=lambda relevance, coherence, fluency: {
            'relevance': relevance,
            'coherence': coherence,
            'fluency': fluency,
            'average': (relevance + coherence + fluency) / 3
        },
        inputs=[
            gr.Textbox(label="Prompt", value=prompt, interactive=False),
            gr.Textbox(label="Response", value=response, interactive=False),
            gr.Slider(1, 5, step=1, label="Relevance"),
            gr.Slider(1, 5, step=1, label="Coherence"),
            gr.Slider(1, 5, step=1, label="Fluency")
        ]
    )
```

```
    ],
    outputs=gr.JSON(label="Evaluation")
)

# Launch
interface = evaluate_response(
    "What is AI?",
    "AI is artificial intelligence..."
)
interface.launch()
```

---

## 2. Collect Ratings

```
import json

class HumanEvalCollector:
    def __init__(self, output_file='evaluations.jsonl'):
        self.output_file = output_file

    def save_evaluation(self, data):
        """Save single evaluation"""
        with open(self.output_file, 'a') as f:
            f.write(json.dumps(data) + '\n')

    def calculate_agreement(self):
        """Calculate inter-annotator agreement (Krippendorff's alpha)"""
        from krippendorff import alpha

        # Load evaluations
        evaluations = []
        with open(self.output_file) as f:
            for line in f:
                evaluations.append(json.loads(line))

        # Calculate agreement
        # ... implementation
        pass
```

---

## Production Monitoring

### Real-time Evaluation

```
from prometheus_client import Counter, Histogram

# Metrics
response_quality = Histogram(
    'llm_response_quality',
```

```

    'Quality scores for LLM responses',
    ['model', 'metric']
)

response_time = Histogram(
    'llm_response_time_seconds',
    'Response generation time'
)

errors = Counter(
    'llm_errors_total',
    'Total LLM errors',
    ['model', 'error_type']
)

def monitored_generate(prompt, model='gpt-3.5-turbo'):
    """Generate with monitoring"""
    import time

    start = time.time()

    try:
        # Generate
        response = openai.ChatCompletion.create(
            model=model,
            messages=[{"role": "user", "content": prompt}]
        )
        generated = response.choices[0].message.content

        # Measure quality (if reference available)
        # quality = evaluate(generated, reference)
        # response_quality.labels(model=model, metric='bert_f1').observe(quality)

        # Measure latency
        elapsed = time.time() - start
        response_time.observe(elapsed)

        return generated

    except Exception as e:
        errors.labels(model=model, error_type=type(e).__name__).inc()
        raise

```

---

## Tools & Platforms

### 1. Commercial Platforms

#### Weights & Biases

```
import wandb

wandb.init(project="llm-eval")

# Log metrics
wandb.log({
    'rouge1': 0.85,
    'bert_f1': 0.90,
    'latency': 1.5
})
```

### LangSmith (LangChain)

- Trace LLM calls
- Auto-evaluate
- Compare runs

### Phoenix (Arize AI)

- Open-source observability
- Trace analysis
- Drift detection

---

## 2. Open-Source Tools

### lm-evaluation-harness

```
pip install lm-eval

# Evaluate on benchmarks
lm_eval --model hf --model_args pretrained=gpt2 --tasks hellaswag,arc_easy
```

### EleutherAI/lm-evaluation-harness

- 60+ benchmarks
- Standardized evaluation

---

## Best Practices

### 1. Use Multiple Metrics

```
def comprehensive_eval(generated, reference):
    return {
        'automatic': {
            'rouge': rouge_score(generated, reference),
            'bert': bert_score(generated, reference),
```

```
        'exact': exact_match(generated, reference)
    },
    'human': {
        'relevance': get_human_rating(generated, 'relevance'),
        'coherence': get_human_rating(generated, 'coherence')
    }
}
```

## 2. Test on Diverse Data

- Edge cases
- Different domains
- Various lengths
- Multiple languages

## 3. Track Over Time

```
import pandas as pd

results = pd.DataFrame(evaluations)
results['timestamp'] = pd.to_datetime(results['timestamp'])

# Plot trends
results.groupby('timestamp')['bert_f1'].mean().plot()
```

## 4. Consider Cost

```
def cost_aware_eval(model, test_cases):
    """Evaluate considering cost"""

    total_tokens = 0
    total_cost = 0

    for case in test_cases:
        response = generate(model, case['prompt'])
        tokens = count_tokens(response)
        cost = calculate_cost(model, tokens)

        total_tokens += tokens
        total_cost += cost

    score = evaluate_batch(test_cases)

    return {
        'score': score,
        'cost': total_cost,
        'cost_per_quality': total_cost / score
    }
```

## Summary

**Quick Reference:**

| Metric      | Use Case            | Range        |
|-------------|---------------------|--------------|
| BLEU        | Translation         | 0-1          |
| ROUGE       | Summarization       | 0-1          |
| BERTScore   | Semantic similarity | 0-1          |
| Perplexity  | Language modeling   | Lower=better |
| Exact Match | Factual QA          | 0/1          |
| Human Eval  | Overall quality     | 1-5          |

**Evaluation Stack:**

1. Automated metrics (fast, cheap)
2. LLM-as-judge (scalable, decent)
3. Human evaluation (slow, expensive, gold standard)

**Tools:**

- LangChain Evaluators
- RAGAS (for RAG)
- TruLens (observability)
- Weights & Biases (tracking)

**Next Steps:**

1. Define your task
2. Choose appropriate metrics
3. Collect test dataset
4. Run evaluations
5. Track over time

Need specific evaluation for your use case? Let me know!